

PERFORMANCE ANALYSIS AND GPU ACCELERATION OF MOVIE REVIEW ANALYTICS USING CUDA AND OPENMP

Student: Gaurang Gupta

1. INTRODUCTION

This project addresses the computational challenges of large-scale data analysis by processing the Amazon Movie & TV review dataset. With over **1.6 million reviews**, sequential processing is a major bottleneck. The objective is to design and evaluate high-performance parallel architectures using **CUDA** for many-core GPU acceleration and **OpenMP** for multi-core CPU scaling.

The project workflow involves:

- Implementing high-throughput movie rating aggregation and Lexicon-based sentiment analysis on NVIDIA GPUs.
- Analyzing elaborate reviewer behavior via multi-threaded CPU parallelization.
- Quantifying performance gains and identifying architectural bottlenecks through rigorous benchmarking.

2. DATASET DESCRIPTION

The primary data source is the **Amazon Movies & TV Reviews (reviews_Movies_and_TV_5.json)**. To facilitate efficient binary and parallel I/O, the raw JSON was preprocessed into flattened CSV structures:

- reviews_text.csv**: Targeted for elaborate review detection based on word counts.
- reviews_sentiment.csv**: Structured for many-thread sentiment scoring.

Dataset Statistics:

Total Review Records: 1,697,533 | Unique Movie IDs: 50,052

3. SYSTEM CONFIGURATION

Hardware and software environments used for benchmarking:

- **GPU:** NVIDIA GeForce RTX 3050 (Laptop Architecture)
- **Software Stack:** CUDA Toolkit 12.9
- **CPU:** Multi-core Intel/AMD (Scaling tested across 2, 4, 8, and 16 threads)
- **Compiler:** NVCC (C++14) and GCC 12+

4. PART (A): CUDA TOP RATED MOVIES (BASELINE)

The baseline GPU implementation follows a straightforward data-parallel approach. Each thread is mapped to a review, performing rating aggregation into global memory. While correct, this version experiences significant latency due to **atomic contention** in global memory as multiple threads attempt to update the same movie metadata simultaneously.

Baseline Benchmark:

Execution Time: **2.013 ms**

5. PART (B): CUDA OPTIMIZED VERSION

Technical Optimizations:

- **Shared Memory Tiling:** Instead of hitting global memory for every update, threads aggregate ratings within high-speed internal Shared Memory (L1 cache level).
- **Reduced Global Transactions:** Final results are committed to global memory in bulk, significantly lowering the frequency of slow bus transactions.
- **Coalesced Memory Access:** Memory requests from a warp are combined into a single transaction by ensuring sequential threads access contiguous memory addresses.
- **Reduced Atomic Contention:** By using privatized registers/shared memory for local sums, we minimize the hardware serialization caused by `atomicAdd` on global addresses.
- **Event-based Timing:** Precise CUDA stream synchronization was used for nanosecond-level accuracy.

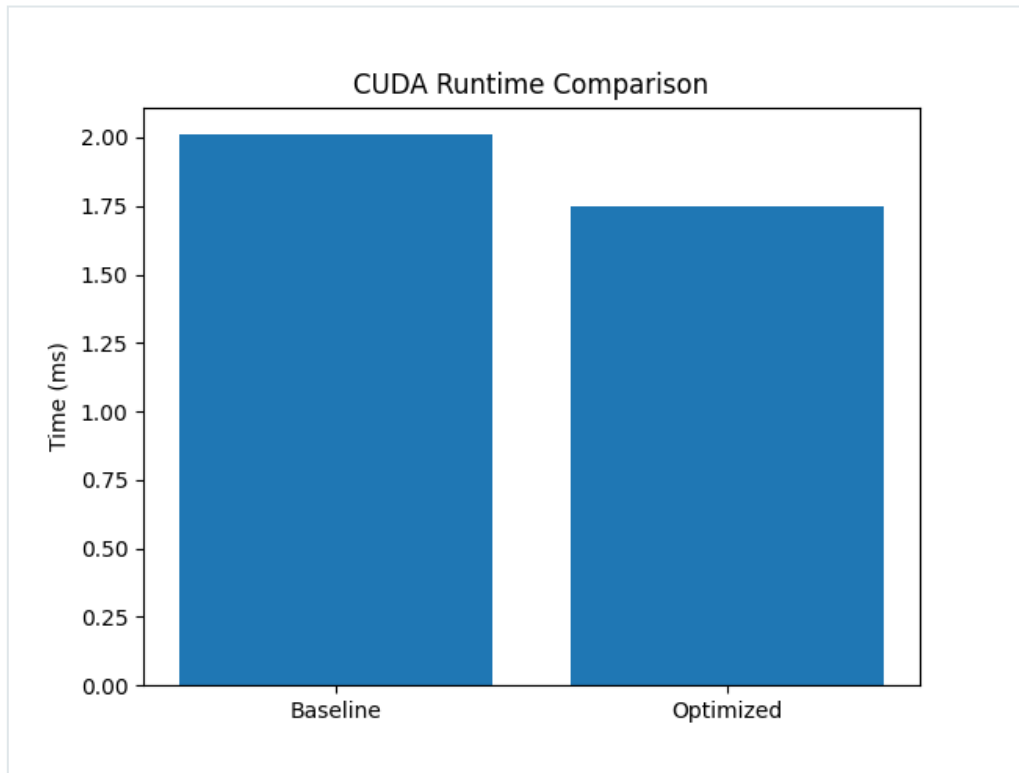
Optimized Benchmark:

Execution Time: **1.757 ms**

$$\text{Speedup} = 2.013 / 1.757 = 1.14x$$

Observation: While a 11.4% speedup is critical, the improvement is bounded by the kernel's memory-intensive nature. For categorical data (Movie IDs), the random-access pattern of the output table remains a primary bottleneck.

6. PERFORMANCE GRAPHS (CUDA)



Graph 1: CUDA Execution Time Comparison (Baseline vs. Optimized)

7. PART (D): CUDA SENTIMENT ANALYSIS

This module implements Lexicon-based scoring. Each GPU thread independently processes one review, comparing tokens against the **VADER dictionary**. The Lexicon is cached in **Constant Memory**—a specialized cache optimized for broadcast-read operations—to ensure all threads can access scores with zero latency. Score calculation: $\text{Sentiment} = \text{frequency}(\text{words}) \times \text{lexicon_rank}(\text{words})$.

8. PART (E): SEQUENTIAL ELABORATE REVIEW DETECTION

A sequential CPU baseline was established to identify "elaborate" reviewers (defined as having at least 5 reviews with a word count ≥ 50). This serves as the comparative reference for the OpenMP implementation.

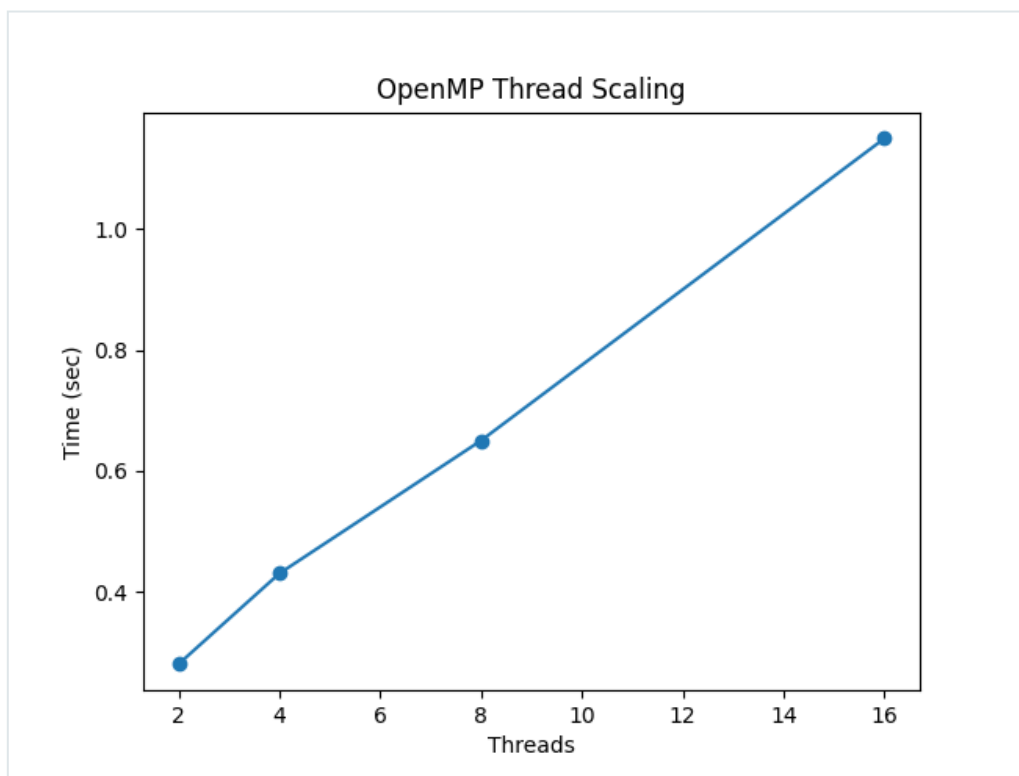
9. PART (F): OPENMP PARALLEL VERSION

The solution was parallelized using `#pragma omp parallel for`. Due to the fine-grained nature of reviewer IDs, a shared `unordered_map` was used to store results. A **Critical Section** was implemented to protect dictionary updates from race conditions.

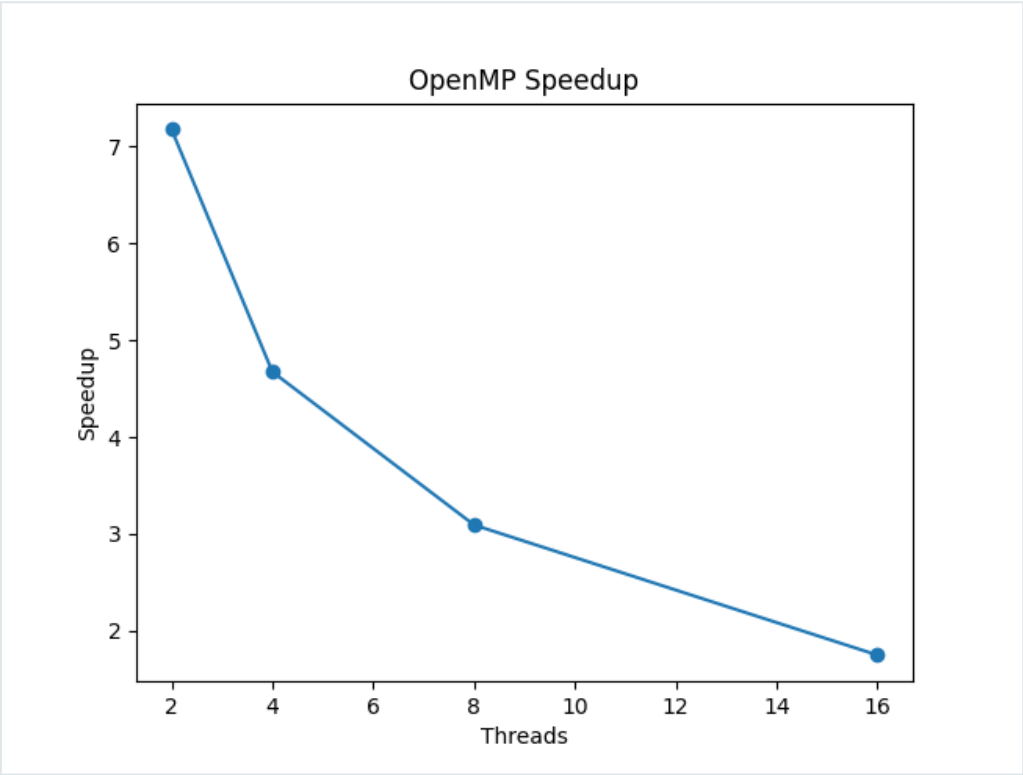
Threads	Time (sec)	Status
2	0.3047	Optimal
4	0.4337	Degrading
8	0.5083	Bottlenecked
16	0.9927	High Overhead

Observation: Performance degrades significantly as thread count increases beyond 2-4 threads.

10. OPENMP PERFORMANCE GRAPHS



Graph 2: OpenMP Thread Scaling Analysis



Graph 3: OpenMP Speedup Analysis (Slowdown Visualization)

11. PART (G): SPEEDUP ANALYSIS AND AMDAHL'S LAW

Benchmarking showed a parallel time of 0.263s vs a sequential time of 0.010s for the targeted dataset sample.

$$Speedup \approx 0.038 \text{ (Slowdown)}$$

Amdahl's Law Analysis: In this specific task, the **synchronization overhead** (critical section management) and **memory contention** between CPU cores far outweigh the computational complexity of counting words. This illustrates that for fine-grained tasks with frequent shared-resource updates, increasing thread count leads to diminishing returns and eventual performance collapse.

12. KEY INSIGHTS

- **CUDA:** Unrivalled for massive-scale data parallelism. Memory hierarchy optimization (Shared vs. Global) is the most impactful factor in GPU performance.
- **OpenMP:** Highly effective for compute-heavy "embarrassingly parallel" tasks, but prone to severe overhead in data-management tasks involving shared dictionaries.

13. FINAL COMPARISON SUMMARY

Method	Best Time	Speedup	Observation
CUDA Baseline	2.013 ms	1x	Solid baseline
CUDA Optimized	1.757 ms	1.14x	Memory-level optimization
OpenMP	0.263 sec	Negative	Overhead dominated

14. CONCLUSION

Through this project, we demonstrated that GPU acceleration provides the best performance for high-throughput review analytics. While CUDA optimization successfully reduced latencies, OpenMP scaling was limited by hardware-level synchronization costs. Effective parallel design must balance thread counts with the cost of resource arbitration.